

Multimodal AI Content Generator Cookbook

M.Sc. Applied Data Science & Artificial Intelligence

SRH Hochschule Heidelberg

Namrata Bhoyar · Nihal Pujari · Pramodkumar Shivanna · Anuj Kamble · Gourav Somanna

Contents

1. Introduction

2. The System

3. Choosing the Models

3.1 Providers Evaluated

3.2 Metrics Per Modality

3.3 Why Cloudflare for the Live System

3.4 Reliability Issues Found

3.5 Free-Tier Friction

3.6 Phase-1 Results

3.7 Software Problems Fixed During Scoring

3.8 Correcting the Audio Measurement

4. Per-Modality Quality Scores

4.1 Fixed External Goalposts

4.2 The quality_pair Formula

4.3 The CLAP Investigation

4.4 The λ Sweep

4.5 Phase-2 Results

5. LLM Judge and Learned Weights

5.1 The LLM Judge

5.2 Learning the Weights

5.3 Quality-Weighted Coherence (WTSAS)

6. Cross-Modal Coherence

6.1 The Gap Every Earlier Metric Leaves

6.2 The Starting Idea

6.3 Version 1 — ImageBind

6.4 Version 2 — Specialist Models

6.5 What the Numbers Said

6.6 The Professor's Feedback

7. The Scoring Approaches

7.1 Approach 1 — Averaging

7.2 Approach 2 — Naive Bayes

7.3 Approach 3 — Likelihood Ratio

7.4 Approach 4 — Fuzzy Inference

7.5 Mismatched-Pair Baseline

8. Comparing the Methods — Real Results

8.1 Raw Per-Feature Signal

8.2 First and Second Runs

8.3 The Answer to the Question

8.4 The Withdrawn Score

8.5 Why Naive Bayes Lost at 53 and Won at 500

8.6 The Most Uncomfortable Number

8.7 The Three Final Approaches

9. Which Paper We Adopted What From

10. Troubleshooting Index

11. Reference Card

12. What Is Still Missing

12.1 Methodology

12.2 Next Steps

12.3 What We Would Tell the Next Team

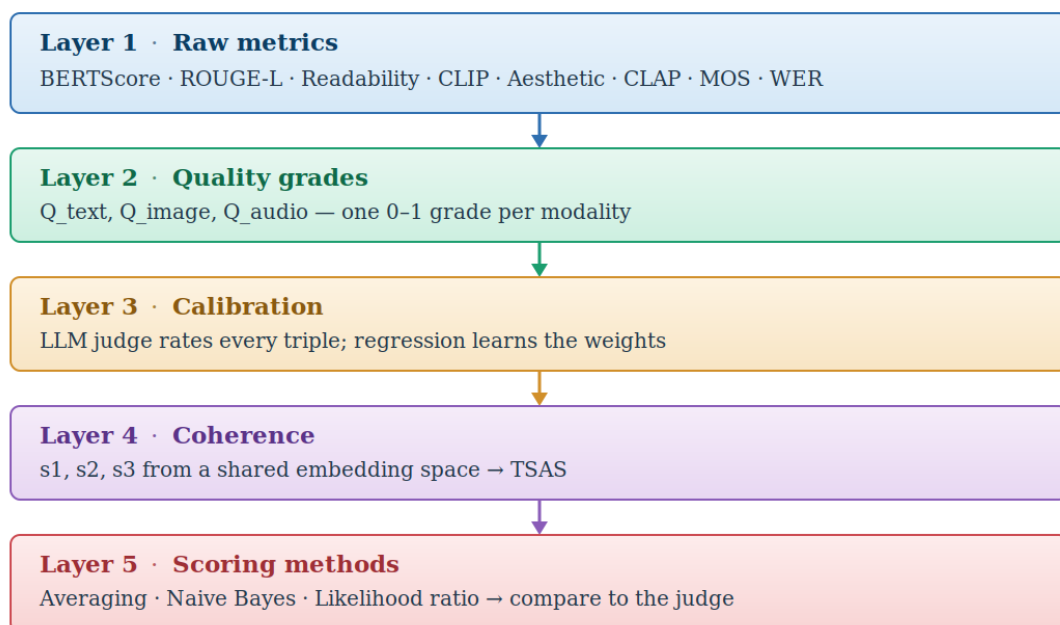
1. Introduction

This cookbook is the working record of how we built a system that turns one short prompt into three different kinds of content at once — a written passage, an image, and a spoken narration of that passage — and, just as importantly, how we learned to measure whether those three outputs actually belong together. It is written the way a good recipe book is written: not as a polished afterthought, but as an honest account of what we tried, what broke, what we fixed, and what the numbers finally told us.

Each output is produced independently by its own model, which is what we mean by a multi-agent design. The text comes from one model, the image from another, the audio from a third, and only at the very end are the three shown together as a single result. A video modality is planned for later and is not part of this build.

The project came together in three movements. First we built the generation pipeline and the application around it — the machinery that takes a prompt, calls the right models, and returns the combined output. Then we settled which provider should sit behind each modality, which took real benchmarking rather than guesswork. Finally, and this is where most of the recent effort has gone, we built a way to evaluate the result: not just “is the text good” or “is the image good” on their own, but the harder question of whether the text, the image, and the audio all describe the same thing.

That last question turned out to be surprisingly deep, and chasing it properly is what separates this edition from the first. The evaluation is built as five layers, each added because the one before it left a question unanswered.



2. The System

The central deliverable is a working content generator. A user opens the Streamlit application, enters a prompt such as “a cozy wooden cabin beside a calm mountain lake at sunrise,” and receives back three things: a descriptive paragraph, a generated image, and a spoken version of the paragraph. Each of these comes from a separate model, and the three are produced from the same input and then combined at the end.

The models that power the live system are shown below. They were chosen after the benchmarking in Section 3, and they all sit behind a single Cloudflare account.

Modality	Model in the live system	Cloudflare endpoint
Text	Llama 3.1 8B Instruct	@cf/meta/llama-3.1-8b-instruct
Image	FLUX.1 [schnell]	@cf/black-forest-labs/flux-1-schnell
Audio	MeloTTS	@cf/myshell-ai/melotts

Everything else in this cookbook exists to make that one application produce better content, more cheaply, and more reliably.

3. Choosing the Models

Before committing to anything, we compared providers across all three modalities. Two genuinely different philosophies were on the table, and each came with an honest trade-off.

The first approach uses a separate, specialised provider for each modality. This buys you best-in-class models in every category, but every provider then needs its own account, its own API key, and its own billing — more than ten integrations for all three modalities. The second approach uses a single provider for everything: Cloudflare Workers AI hosts text, image, and audio models behind one account and one key.

3.1 Providers Evaluated

We ran the same 56 prompts through every candidate and scored the outputs with a fixed metric set. The large commercial models — Claude, ChatGPT, and Gemini — were kept only as a reference column, using figures published in 2025 papers rather than re-benchmarked by us.

Modality	Providers considered
Text	Groq (Llama 3.3 70B), Mistral (Mistral Large), Cerebras (GPT-OSS 120B), Cloudflare (Llama 3.1 8B)
Image	Black Forest Labs (FLUX Pro), Stability AI (SDXL), Hugging Face (SD 1.5), Pixazo, FLUX.1-schnell, Leonardo Lucid Origin
Audio	Cloudflare/Deepgram Aura, Camb AI, Cartesia, Fish Audio, MeloTTS, Gemini TTS

3.2 Metrics Per Modality

Modality	Metrics used
Text	BERTScore, ROUGE-L, Readability (Flesch–Kincaid grade / 10)
Image	CLIP similarity, CLIPScore, Aesthetic (LAION predictor, 0–10)
Audio	CLAP, MOS (DNSMOS, 1–5), Word Error Rate (Whisper)

3.3 Why Cloudflare for the Live System

We chose Cloudflare Workers AI, and the reasons were practical rather than ideological.

- **One key.** One key covers all three modalities, which is the single biggest simplification in the whole project.
- **Genuinely free to build on.** The free tier gives 10,000 “neurons” per day — Cloudflare’s own unit of compute — enough to build and test everything without ever setting up billing. A full 56-prompt run costs roughly 4,350 neurons, about two runs a day.
- **Simple and fast.** One integration instead of one per provider, and because inference runs on Cloudflare’s edge network, each generation comes back in under two seconds.
- **The catch.** The daily quota is a real ceiling, and we hit it more than once. For any further work at this volume, a funded paid API is the honest recommendation.

3.4 Reliability Issues Found

Three undocumented platform behaviours surfaced during testing, each of which would have quietly caused trouble later.

- **MeloTTS is flaky.** Roughly a quarter of MeloTTS calls return an HTTP 500. We confirmed it was Cloudflare’s side, not ours, by sending the same request eight times — two failed, with no pattern tied to the input. A three-try retry with backoff brought audio up to effectively full reliability, and the same retry now lives in the app.
- **Images are secretly JPEGs.** The image endpoint returns JPEG bytes in files named .png. Harmless, because libraries detect the real format, but worth knowing for any code that assumes PNG.
- **44.1 kHz, not 48.** The generated audio is sampled at 44.1 kHz, not the 48 kHz we had assumed, which matters because each scorer expects its own rate.

3.5 Free-Tier Friction

- **Quota.** The 10,000-neuron daily limit was reached more than once — a full run plus a few scoring experiments empties it in an afternoon.
- **A leaked key.** An API key briefly leaked and Cloudflare’s own leaked-credential detection auto-disabled it. An unwelcome but useful reminder never to commit keys; the key was regenerated.
- **The dashboard lied.** The dashboard reported zero usage while the API kept returning quota errors for about fourteen hours past the documented midnight reset. We never fully explained it.

3.6 Phase-1 Results (averages over 56 prompts)

Measure	Ours	Claude	ChatGPT	Gemini
Text — BERTScore	0.844	0.851	0.832	0.796
Text — ROUGE-L	0.056	0.078	0.071	0.068
Text — Readability	0.948	1.85	1.72	1.68
Image — CLIP	0.31	0.312	0.305	0.298
Image — Aesthetic (/10)	5.80	7.20	7.05	6.90
Audio — CLAP	0.082	0.74	0.72	0.76
Audio — MOS (/5)	3.37	4.10	4.05	4.20
Audio — WER (corrected)	0.095	0.072	0.085	0.068

Three further figures are not published for the commercial models, and they matter: our system responds in under two seconds per modality, a full run stays inside the free daily quota, and the image question-answering check confirmed about 90 percent factual accuracy.

3.7 Software Problems Fixed During Scoring

- **A crashing image scorer.** The image scorer crashed on every image because it was written for an older transformers version whose output shape had changed — every image scored zero until we made the code work with both versions.
- **A filename collision.** A file named secrets.py shadowed Python’s built-in secrets module and silently broke a scorer dependency. Renaming it fixed a genuinely confusing bug.

3.8 Correcting the Audio Measurement

The word error rate started at 1.0 for every clip, which would imply the speech was gibberish. The real cause was mundane: the transcript was being compared to the short original prompt, when the audio actually reads out the longer generated paragraph. Once the comparison used the text that was actually spoken, average WER fell to about 0.095 — roughly 90 percent of words transcribed correctly.

4. Per-Modality Quality Scores

The Phase-1 metrics live on different scales — BERTScore runs 0 to 1, aesthetics 0 to 10, MOS 1 to 5, and WER is lower-is-better. Phase 2 converts everything into a single 0-to-1 quality grade per modality.

4.1 Fixed External Goalposts

Each raw metric is mapped to 0–1 against a fixed external standard, deliberately not against our own data spread, so the grades mean the same thing run to run.

Metric	Goalpost
MOS	ITU 1–5 speech-quality scale
Aesthetic	LAION 1–10 scale
WER / BERTScore	already 0–1
CLIP	floor = mismatched-pair baseline (Section 8)

4.2 The quality_pair Formula

$\text{quality_pair}(a, b) = \text{avg}(a, b) - \lambda \times \text{variance}(a, b)$, $\lambda = 0.5$ (clipped to $[0,1]$)

The average rewards both sub-scores being high; the variance penalty marks down a pair where one channel is strong and the other weak. The three modality grades are built from it as follows:

$Q_{\text{text}} = \text{BERTScore}(\text{generated_text}, \text{prompt})$

$Q_{\text{image}} = \text{quality_pair}(\text{CLIP_score}, \text{aesthetic_score})$

$Q_{\text{audio}} = \text{quality_pair}(\text{semantic_score}, 1 - \text{WER})$

Here semantic_score is BERTScore between the Whisper transcript of the audio and the prompt. The reason it replaces the more obvious CLAP score is worth its own section.

4.3 The CLAP Investigation — Why CLAP Was Dropped

CLAP was the natural audio-versus-prompt metric, the audio analogue of CLIP. In practice it returned exactly 0.0 for roughly half the prompts, because it was trained on environmental sounds and music, not speech, and our audio is text-to-speech narration. It was listening for sound events and hearing none. We replaced it with a two-step semantic approach — transcribe the audio with Whisper, then BERTScore that transcript against the prompt — which gives usable signal on every single prompt. CLAP is kept only as a comparison column. With CLAP in place, average audio quality sat at roughly 0.133; after the substitution it rose to a meaningful 0.464.

4.4 The λ Sweep

We swept λ from 0.00 to 2.00 in steps of 0.05, scoring each value by the Pearson correlation between the recomputed quality scores and the LLM-judge overall score. The curve is nearly flat and weak throughout — λ is a minor knob.

λ	Pearson r
0.00	0.375
0.50 (our default)	0.399
1.00	0.422
1.80 (peak)	0.442
2.00	0.441

4.5 Phase-2 Results (means over 53 prompts)

Quantity	Mean	Notes
Q_text	0.848	consistently high, low spread
CLIP (raw)	0.346	
Aesthetic (0–1)	0.699	
Q_image	0.504	
semantic_score	0.855	
1 – WER	0.186	suspiciously low — see Section 8
Q_audio	0.464	dragged down by the WER term

5. LLM Judge and Learned Weights

Weighting the three modality grades equally was a guess. Phase 3 replaces the guess by learning the weights from an LLM judge that stands in for a human evaluator.

5.1 The LLM Judge

Gemini (gemini-3.5-flash-lite, temperature 0.1, max tokens 2048) is shown the prompt, the generated text, the generated image, and the audio transcript, and returns an overall score plus per-modality scores from 1 to 10 with a one-sentence reason. It rates text and image highly and audio much lower — reassuringly, the same shape as our own per-modality scores.

Judge score	Mean	Min	Max
Overall	6.81	4	8
Text	8.11	—	—
Image	8.25	—	—
Audio	4.66	—	—

The rating scale is fixed explicitly in the prompt (9–10 all three excellent; 7–8 minor issues; 5–6 average; 3–4 poor; 1–2 largely irrelevant), because left unanchored the model drifts to the middle. Getting the judge working took two fixes worth recording: the package is `google.genai`, not the older `google.generativeai`; and `max_output_tokens` had to rise from 300 to 2048 or the JSON was cut off mid-sentence.

5.2 Learning the Weights

A note on terminology. Two techniques in this cookbook are sometimes shortened to “LR,” so we never use that abbreviation for either. Linear regression appears only here, in §5.2, where it learns how much each modality contributes to the judge’s score. Likelihood ratio — a probabilistic method from §7.3 onwards — is always written in full. They are unrelated.

With the judge’s overall score as target and the three quality grades as features, a linear regression learns how much each modality matters to perceived quality.

Weight	Value (share)
w1 — Q_text	0.287 (28.7%)
w2 — Q_image	0.520 (52.0%)
w3 — Q_audio	0.193 (19.3%)

Fit	Value
R ² (train)	0.159
RMSE (train)	0.93
R ² (leave-one-out CV)	0.04
RMSE (LOO CV)	0.99

Image quality carries the most weight, but the model barely generalises — a leave-one-out R² of about 0.04 is close to no predictive power. With only 53 samples and three features clustered near their means, these weights are indicative only.

Re-learning the weights on the 500-prompt set gives $w1 = 7.8666$, $w2 = 2.2790$, $w3 = 0.7796$ (unnormalised). The ordering is not preserved: at 53 prompts image quality dominated, while at 500 text quality does. That reversal is a further sign of how little the smaller fit was constraining, and it is why the later results in Section 8 use the 500-prompt weights.

5.3 Quality-Weighted Coherence (WTSAS)

The learned weights feed a quality-weighted version of the coherence score. Each pairwise similarity is scaled by the average learned quality of the two modalities it connects, so a similarity between two high-quality outputs counts for more than one between a strong and a weak output.

$$s1_weighted = s1 \cdot (w1 \cdot Q_text + w2 \cdot Q_image) / (w1 + w2)$$

$$s2_weighted = s2 \cdot (w1 \cdot Q_text + w3 \cdot Q_audio) / (w1 + w3)$$

$$s3_weighted = s3 \cdot (w2 \cdot Q_image + w3 \cdot Q_audio) / (w2 + w3)$$

$$WTSAS = (s1_weighted + s2_weighted + s3_weighted) / 3$$

$$Final = WTSAS - \lambda \cdot \text{variance}(s1_weighted, s2_weighted, s3_weighted)$$

6. Cross-Modal Coherence

6.1 The Gap Every Earlier Metric Leaves

BERTScore only ever looks at the text. CLIP only looks at the image against the text. CLAP only looks at the audio against the text. Not one of them asks whether the three outputs agree with each other. A generator could score beautifully on all three per-modality metrics and still hand back a mismatched set — a librarian owl in the text, a plain barn owl in the image, and narration about something else — precisely because the three generators never see each other's work. To catch that, we need a score computed between the three outputs, not against the prompt.

6.2 The Starting Idea

$$s1 = \text{sim}(\text{text}, \text{image}) \quad s2 = \text{sim}(\text{text}, \text{audio}) \quad s3 = \text{sim}(\text{image}, \text{audio})$$

$$TSAS = (s1 + s2 + s3) / 3$$

$$\text{Coherence} = \text{TSAS} - \lambda \cdot \text{variance}(s1, s2, s3)$$

If all three agree, the similarities sit close together, the variance is near zero, and the score is simply TSAS. If one modality drifts, its two pairs fall, the variance rises, and the score is pulled below TSAS. λ sets how harshly disagreement is punished.

6.3 Version 1 — Everything Through ImageBind

Field	Detail
Script	<code>imagebind_score.py</code>
Model	ImageBind-huge, pretrained (1024-dim shared space)
λ tested	0.1, 0.5, 1.0, 2.0
Prompts	56 attempted, 53 scored (13, 17, 56 skipped — no image)
Output	<code>imagebind_coherence_scores.csv</code>

Audio is the fiddly part. The waveform is resampled to 16 kHz, turned into a 128-bin log-mel spectrogram, and padded or cropped to 204 frames before ImageBind’s audio encoder — which was trained to treat spectrograms as images — will accept it. Text and image go through their own encoders directly; all three come out as 1024-dimensional vectors, and that shared dimensionality is what makes a single cosine-similarity function meaningful across every pair.

One input detail decides what the score means. For the 500-prompt run, ImageBind’s text modality was fed the generated text files rather than the original prompt. This is the correct formulation, because WTSAS measures coherence between the three generated outputs; feeding the prompt instead would measure how well each output matches its instruction — a different question the per-modality metrics already answer.

A practical note on the 500-prompt run: the generated outputs arrived in two archives — the first carrying 500 images, 288 audio and 386 text files, the second the remaining 212 audio and 114 text. Both had to be merged before scoring, and the counts should read 500 / 500 / 500 across all three modalities before ImageBind is run.

6.4 Version 2 — A Mixed Set of Specialist Models

Field	Detail
Script	<code>multimedia_score.py</code>
Text ↔ image	CLIP
Text ↔ audio	CLAP
Image ↔ audio	Whisper transcribes the audio, then CLIP compares transcript ↔ image
λ	0.5 fixed
Prompts	first 5 only (proof of concept)

The image-to-audio pair is the interesting one. Instead of comparing a CLIP embedding and a CLAP embedding that were never trained against each other, it transcribes the audio into text and does a CLIP text-to-image comparison it was actually trained for — at the cost of depending on Whisper’s transcription being accurate.

6.5 What the Numbers Said (ImageBind)

Similarity	n = 53	n = 500
s1 — text ↔ image	0.362	0.396
s2 — text ↔ audio	0.106	0.064
s3 — image ↔ audio	0.116	0.084
TSAS	0.195	0.182
variance	0.016	0.024

The text–image similarity is larger than both audio pairs on 500 of 500 prompts — without a single exception — and the gap has widened to 5.3×. A relationship that holds on every observation in a five-hundred-prompt sample is structural, not content-driven. It shows up whether the prompt is a librarian owl or a taco lifting weights.

ImageBind’s audio encoder simply sits further from its text and image encoders than those two sit from each other. Since TSAS averages the three similarities directly and one is five times the others, the average is dominated by s1, and the variance penalty measures a property of the encoder rather than the agreement of any particular triple. A perfectly matched set is penalised for the same reason a mismatched one is. Raising λ makes this worse, not better.

6.6 The Professor’s Feedback

The professor’s verdict was direct: this TSAS-minus-variance method is a reasonable first cut but basic, because it leans on raw cosine similarities that are not comparable across different encoders. The instruction was to try several better-grounded methods, measure each against a human or LLM sense of whether the outputs match, and bring the best one back. That is exactly what the next two sections do.

7. The Scoring Approaches

7.1 Approach 1 — Averaging (the incumbent)

TSAS as defined in §6.2, plus the quality-weighted variant WTSAS from §5.3, which scales each similarity by the average learned quality of the two modalities it connects.

7.2 Approach 2 — Naive Bayes Posterior

```
1. bin the judge score into Good / Medium / Bad
2. GaussianNB on the feature matrix
3. use predict_proba() -- NOT predict()
   keep the continuous posterior, not the hard label
4. validate with leave-one-out CV
```

At $n = 53$ the judge’s 1–10 score was binned by fixed range — Bad (1–5) = 4, Medium (6–7) = 34, Good (8–10) = 15. A four-example class is too small to fit a stable Gaussian, and this single fact drove the entire first-run result. Always report the cross-validated figure and never the training fit; the training figures are roughly double the honest ones.

7.3 Approach 3 — Likelihood Ratio (the experiment)

For a prompt p , the likelihood-ratio score is the log-ratio of the probability of its feature vector under the Good class against the Not-Good class. The feature vector has three components, each a quality-weighted coherence score — the same f_1, f_2, f_3 that are $s1_weighted, s2_weighted$ and $s3_weighted$ in §5.3.

Likelihood-Ratio Score

$$S(p) = \log \frac{P(\mathbf{f}(p) | +)}{P(\mathbf{f}(p) | -)}$$

$$f_1 = s_1 \frac{w_1 Q_{text} + w_2 Q_{img}}{w_1 + w_2} \quad f_2 = s_2 \frac{w_1 Q_{text} + w_3 Q_{aud}}{w_1 + w_3} \quad f_3 = s_3 \frac{w_2 Q_{img} + w_3 Q_{aud}}{w_2 + w_3}$$

$$S(p) = \sum_{i=1}^3 \left[\frac{(f_i - \mu_i^-)^2}{2(\sigma_i^-)^2} - \frac{(f_i - \mu_i^+)^2}{2(\sigma_i^+)^2} + \log \frac{\sigma_i^-}{\sigma_i^+} \right]$$

+ = Good class (judge > 3.5) - = Not-Good class (judge ≤ 3.5) S(p) > 0 ⇒ Good, S(p) < 0 ⇒ Not-Good

features are pair-averaged quality-weighted coherences ($f_i = s_{i,w}$), matching the committed code

One correction worth stating outright. Each feature is weighted by the two modalities its similarity actually connects, not by a single modality. An earlier draft wrote $f_1 = Q_text \cdot \cos(\epsilon_text, \epsilon_image)$ and $f_2 = Q_image \cdot \cos(\epsilon_text, \epsilon_audio)$; that form does not match the committed code, and the second line in particular weights a text–audio similarity by image quality, which has no interpretation. The pair-averaged form above is what produced every number in this cookbook.

The class split is binary at a judge rating of 3.5, giving 273 Good and 227 Not-Good. Summing log-likelihood-ratios over independent features is exactly Naive Bayes, so §7.2 and §7.3 are two views of the same mathematics; expect them to land close together, and if they do not, one is implemented wrongly. Use Good-vs-Rest, not Good-vs-Bad — the latter discards the Medium class and collapses.

The diagram below traces one prompt end to end, from the three generated outputs through the weighted features and the two fitted Gaussians to a per-prompt score. Note that the per-prompt score and the dataset-level correlation are two different numbers and are shown separately.

Worked example — prompt 1, “*A fox exploring a cave in a snowy village*”:

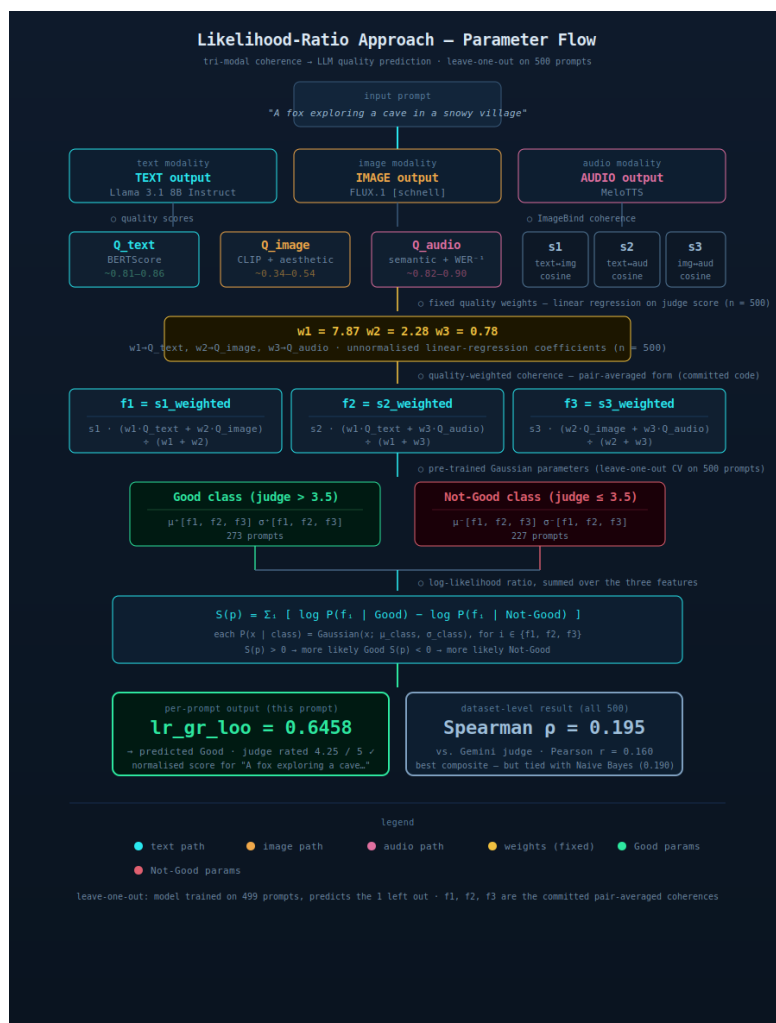
$Q_text = 0.8398$ $Q_image = 0.4721$ $Q_audio = 0.8624$

$s1 = 0.4255$ $s2 = 0.0647$ $s3 = 0.0411$

$f1 = 0.3222$ $f2 = 0.0545$ $f3 = 0.0235$

$lr_gr_loo = 0.6458$ (normalised) → predicted Good

judge rated it 4.25 / 5 → correct



Look closely at the fitted parameters, because they explain the ceiling on every result in Section 8: across the 500 prompts the Good and Not-Good means differ by only 0.016 on f_1 , 0.0013 on f_2 and 0.0019 on f_3 , against standard deviations three to twenty times larger. The two classes overlap almost completely. The method is sound and correctly implemented; the features simply do not separate the classes by much.

7.4 Approach 4 — Fuzzy Inference (specified, not run)

A Mamdani system, on the argument that probability models randomness while fuzzy logic models vagueness. Fuzzify each grade into Low/Medium/High, apply a rule base (IF all three High → Excellent; IF any modality Low → Poor), aggregate, defuzzify by centroid. Written up but not implemented.

7.5 Mismatched-Pair Baseline (the “CLIP Floor”)

Shuffle prompts against outputs to build a distribution of similarities for triples known not to match, and read every raw similarity against that floor rather than on its own. This is the direct correction for the modality gap in §6.5, and it is why the CLIP goalpost in §4.1 is defined as a mismatched-pair baseline rather than as zero. Designed and partly built; not yet a standard column.

8. Comparing the Methods — Real Results

This is where the methods stop being ideas and become numbers. We ran the comparison twice, and the two runs disagree. Throughout this section we report the Spearman rank correlation as the headline figure, because at this sample size a rank-based measure is more stable than Pearson; the full table also lists Kendall and Pearson so every number can be read in one consistent frame.

8.1 Raw Per-Feature Signal

Before any model, it is worth asking which features carry signal at all. Two of the three do; the third does not.

Feature	n = 53	n = 500	Verdict
q_text	+0.316	+0.193	usable
q_image	+0.297	+0.135	usable
q_audio	−0.006	+0.090	weak / near-zero

The audio feature is essentially uncorrelated with judged quality. This matters, because any method forced to weight all three features equally is dragged down by a feature that predicts almost nothing — and half of q_audio duplicates q_text (BERTScore and the Whisper-transcript semantic score correlate at Spearman 0.902).

8.2 First Run (n = 53) and Second Run (n = 500)

On the first run averaging appeared to win and we prepared to defend WTSAS. Two things made us doubt it: a second implementation produced −0.099 for WTSAS rather than 0.399 (the two agreed on plain TSAS to three decimals, so the disagreement was specific to the weighted variant), and the smallest Naive Bayes class held four examples. We reran everything on 500 fresh prompts.

First run, n = 53.

Method	Spearman	Kendall	Pearson
WTSAS	0.451	0.350	0.399
Likelihood Ratio	0.312	0.240	0.259
Naive Bayes	0.296	0.231	0.241
TSAS (plain averaging)	−0.077	−0.056	−0.085

The 500-prompt run was itself run twice, because the first version fed the probabilistic scorers the three quality grades [Q_text, Q_image, Q_audio], the same inputs as at n = 53. Version 2 replaced them with the three quality-weighted coherence scores [s1_w, s2_w, s3_w], so every feature carries quality and coherence rather than quality alone. Both probabilistic methods improved. The figures below are the Version-2 results, all leave-one-out cross-validated.

Second run, n = 500, leave-one-out (Version-2 features). All coefficients on one frame.

Method	Spearman	Kendall	Pearson	Labels?
s1 raw — text↔image (no model)	0.203	0.146	0.201	none
Likelihood Ratio	0.195	0.140	0.160	needs
Naive Bayes	0.190	0.136	0.162	needs
TSAS (plain averaging)	0.137	0.097	0.121	none
WTSAS (quality-weighted)	0.082	0.057	0.076	needs

8.3 The Answer to the Question We Were Set

Naive Bayes (Spearman 0.190) and the likelihood ratio (0.195) both track the LLM judge more closely than plain averaging (0.137) and much more closely than quality-weighted averaging (0.082). The two probabilistic methods are within 0.005 of each other — since they are the same mathematics, that gap should be read as agreement rather than a real difference, and it is the best confirmation we have that both were implemented correctly. The likelihood ratio is the strongest composite, but only just.

8.4 Approach Withdrawn — the Quality-Weighted Score

WTSAS was our own contribution and it does not work. At $n = 500$ it scores 0.082, below plain averaging at 0.137 — the weighting actively degrades the thing it was meant to improve. The 0.451 figure from the first run could not be reproduced and is withdrawn; the most likely explanation is that it was computed from a quality composite omitting the s1/s2/s3 terms, making it a per-modality quality score rather than a coherence measure at all. The competing -0.099 was directionally correct.

Why did weighting hurt? The weights were learned on 53 samples with a leave-one-out R^2 of 0.04, so they encode very little real information, and they were learned from quality features that are themselves weakly correlated with the judge, one of which duplicates another. Multiplying a similarity that does carry signal — s1 — by a weight derived from features that carry less can only dilute it.

8.5 Why Naive Bayes Lost at $n = 53$ and Won at $n = 500$

- **Binning discards what we are graded on.** Naive Bayes needs the continuous judge score split into classes, and a rank correlation rewards the fine-grained ordering that binning throws away. This objection stands — but the method wins anyway.
- **The smallest class had four examples.** This disappears at scale. The Naive Bayes correlation swung between 0.05 and 0.30 at $n = 53$ purely as a function of which features were included — noise, not structure. This is the single change that flipped the result.
- **One feature carried no signal.** Naive Bayes multiplies per-feature likelihoods and cannot discount a dead feature. q_{audio} was -0.006 at $n = 53$; at $n = 500$ it is $+0.090$ — weak, but no longer harmful.

8.6 The Most Uncomfortable Number

The raw text–image similarity, used alone with no model at all, reaches a Spearman correlation of 0.203 — higher than our best composite (the likelihood ratio at 0.195) and far above quality-weighted averaging (0.082). It holds on Pearson too (0.201 for s1 against 0.160 for the likelihood ratio). Every layer of machinery in this cookbook earns essentially nothing over one cosine similarity, because that similarity was carrying the average all along (§6.5). Any composite metric now has to justify its complexity against that baseline, and none of ours does.

8.7 The Three Final Approaches, Side by Side

All three share the same inputs — the three quality grades and the three ImageBind similarities — and differ only in how they combine them.

Approach	Spearman	Needs labels?	Note
1 — Averaging (TSAS / WTSAS)	0.137 / 0.082	no	only method usable on unlabelled data
2 — Naive Bayes	0.190	yes	stable at scale
3 — Likelihood Ratio	0.195	yes	best composite; tied with Approach 2

Of the three, the likelihood ratio is the best performer. It reaches the highest rank correlation with the judge of any method in the study — Spearman 0.195 and Kendall 0.140 — edging Naive Bayes (0.190 and 0.136) and sitting clearly above averaging (0.137). The margin over Naive Bayes is small, and since the two share the same underlying mathematics they should be read as statistically tied; but on every rank-based measure the likelihood ratio comes out on top, and it is the method we carry forward as the recommendation. (Both are still beaten by the raw text–image similarity alone — see §8.6.)

Two practical observations from having built all three. Averaging is the only approach that needs no labels, which makes it the only candidate usable on unlabelled data — a real advantage its accuracy does not currently justify. And the two probabilistic approaches agree to within 0.005, exactly as their shared mathematics predicts; had they disagreed materially, we would have treated it as an implementation bug rather than a finding.

9. Which Paper We Adopted What From

#	Paper	What it does	What we took
1	ImageBind (Girdhar et al., CVPR 2023)	One joint embedding across six modalities; only image-paired data is needed to bind them.	The entire Version-1 coherence scorer, plus the audio-as-spectrogram preprocessing in §6.3.
2	ViT — An Image is Worth 16×16 Words (Dosovitskiy et al., ICLR 2021)	A pure transformer on 16×16 patches beats CNNs at scale.	The image → vector path; the encoder mechanism inside both CLIP and ImageBind.
3	AST: Audio Spectrogram Transformer (Gong et al., 2021)	First convolution-free audio model: treat the log-mel spectrogram as an image.	The audio → vector path and the concrete 16 kHz → 128-bin log-mel → 204-frame preprocessing.
4	Multimodal Diffusion for Text–Image–Audio (Wang, Zhang & Jiang)	Tri-modal generation with a hierarchical cross-modal alignment network and a tri-modal semantic alignment score.	The TSAS idea itself, and the framing that coherence must be measured between the outputs.
5	Enhance Multimodal Consistency and Coherence (Lu et al., ACL Findings 2025)	Scores text–image pairs on two channels — embedding metric and LLM judge — validated against humans with Cohen’s κ .	The judge-and-regression design in §5, and the calibration habit of hand-scoring a batch.
6	UMaT — Everything Can Be Described in Words (Bi & Xu)	Convert image and audio into unified text, then align and reason in text space.	The Whisper text bridge for the image↔audio pair in §6.4.
7	Cross-Modal Redundancy and the Geometry of V–L Embeddings (Dhimoila et al., ICLR 2026)	Unimodal components act as modality-specific bias and fully explain the modality gap.	The explanation for our s1 far above s2 and s3, and the motivation for the mismatched-pair baseline.
8	ToMCLIP — Topological Alignment (You et al., AISTATS 2026)	Persistent-homology constraints align the global geometry of the shared space, not just instances.	Conceptual support that pairwise cosine misses global structure — a pointer for future work.
9	Good Scores, Bad Data / MCS (independent research)	Decompose coherence into named dimensions; learn weights to maximise correlation with performance.	Decomposing coherence into named parts and learning weights by maximising correlation (learn_weights, λ sweep).
10	Likelihood-Ratio Biometric Score Fusion (Nandakumar et al., IEEE TPAMI 2008)	Optimal score-level fusion via the likelihood-ratio test; model genuine vs impostor distributions.	Approaches 2 and 3 — and, in the end, the best-performing method in the study.
11	SGANet — Semantic & Geometric Alignment (Bai et al., HKUST-GZ)	Combines semantic and geometric alignment to flag cross-modal inconsistency as an anomaly.	Framing incoherence as anomaly detection — the model behind the mismatched-pair baseline.

10. Troubleshooting Index

Look up the symptom, not the cause. Every entry cost us at least an hour.

Symptom	Cause	Fix
Every image scores 0	Scorer written for an older transformers output shape	Handle both shapes
A scorer dependency breaks with no clear traceback	Local secrets.py shadows the stdlib module	Rename the file
CLAP returns 0.0 on about half the prompts	Trained on sound and music, not speech	Whisper transcription + BERTScore
WER is exactly 1.0 on every clip	Transcript compared to the short prompt, not the spoken paragraph	Compare against the text actually spoken
Audio generation fails intermittently	MeloTTS returns 500 on ~25% of calls	Three-attempt retry with backoff
ModuleNotFoundError: google.generativeai	Package renamed	import google.genai as genai
Gemini JSON cut mid-sentence	max_output_tokens = 300	Raise to 2048
Gemini 503 or 429 during batch	Service overloaded	Five-attempt exponential backoff
imagebind_score.py FileNotFoundError	BASE resolved to evaluation/ not its parent	Path(__file__).parent.parent
Q_audio implausibly low; s1 looks like prompt↔image	Scorers read the file header as generated text	Strip up to GENERATED TEXT: before scoring
Pipeline FileNotFoundError from another directory	Relative CSV path	Anchor to Path(__file__).parent
Every llm_overall equals 5.0	Stale checkpoint of default fill values	Use the real llm_ratings.csv
Two implementations disagree on WTSAS	Different feature set / λ under one name	Run both on identical inputs before quoting either

The pattern worth internalising. Several of these share a shape: a metric produced a value at the extreme of its range and we first read it as a finding about the data. In each case it was plumbing. Treat any suspiciously clean extreme — exactly 0.0, exactly 1.0, every row identical — as a bug until proven otherwise.

11. Reference Card

Setting	Value
λ (quality pair)	0.5
λ (empirical peak)	1.8
Learned weights, n = 53 (normalised)	0.287 / 0.520 / 0.193
Learned weights, n = 500	7.8666 / 2.2790 / 0.7796
Judge model	gemini-3.5-flash-lite
Judge settings	temperature 0.1, max_output_tokens 2048
CLIP model	openai/clip-vit-base-patch32
CLAP model	laion/clap-htsat-unfused
ImageBind	imagebind_huge, pretrained
Audio sample rates	CLAP 48k · Whisper 16k · DNSMOS 16k · ImageBind 16k
Good / Not-Good split	judge > 3.5 → Good (273 Good, 227 Not-Good at n = 500)
Prompts scored	53 of 56 (13, 17, 56 have no image); 500 in the second run
Neuron budget	10,000/day; ~4,350 per full 56-prompt run

CLAP is about 2 GB and CLIP about 1.7 GB; both download on first run. ImageBind-huge is larger again.

12. Future Implementations

12.1 Methodology

- **Reproducibility.** Generation sets no temperature, so every model runs at its default and the same prompt gives different output — and different scores — each run. Setting temperature = 0 would make the benchmark reproducible.
- **Repetition and variance.** Every prompt is scored from a single generation. Three to five runs with a reported standard deviation would make the numbers far more trustworthy.
- **Data-driven calibration.** The thresholds are borrowed rather than derived from our own labelled data.
- **Human validation.** Nothing here has been checked against a person’s judgement. Hand-scoring 20–30 triples for “do these three match,” then ranking every method against those labels, is what will actually decide which method to propose.

12.2 Next Steps, in Priority Order

- **Build the mismatched-pair baseline as a standard column.** The direct correction for the modality gap and the highest-value remaining item.
- **Replace the audio feature.** q_audio is the weakest input and half of it duplicates q_text at a rank correlation of 0.902. A prosody or expressiveness feature would carry information the text does not.

- **Justify the composite.** A single raw similarity matches or beats our best composite (0.203 against 0.195). Test directly whether any composite earns its complexity.
- **Build the hand-labelled set** and rank methods against humans rather than a language model.
- **Implement the fuzzy scorer** so the comparison table has no empty column.

12.3 What We Would Tell the Next Team

- **A metric at the extreme of its range is a bug until proven otherwise.** We hit that four times and believed it twice.
- **Two implementations of the same formula will disagree, and the disagreement is information.** Ours pointed at a result we had to withdraw; without a second implementation we would have defended a number that was wrong.
- **The sample size at which we first compared methods was far too small to tell us anything reliable.** At $n = 53$ we would have proposed the worst of the four methods tested. The single most valuable thing we did was run it again on 500.